

Code for you to experiment with

Use the code cell below as a starting point. Train at least 3 different CNN models by changing augmentation, regularization, and/or model capacity. Examples include adding rotation, zoom, L2 regularization, dropout, or another convolutional layer. Compare training, validation, performance, and explain which model generalizes best.

[Generate](#) [Code](#) [Markdown](#)

```
# Try different augmentation choices
data_augmentation = models.Sequential([
    layers.RandomFlip("horizontal"),
    # layers.RandomRotation(0.05),
    # layers.RandomZoom(0.05),
    # layers.RandomTranslation(0.05, 0.05),
    # layers.RandomContrast(0.1)
])

# Early stopping
early_stopping_aug = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)

# Set random seed for reproducibility
tf.random.set_seed(42)

# Define CNN model with experimentation options
cnn_aug_catdog = models.Sequential([
    layers.Input(shape=(128, 128, 3)),
    data_augmentation,

    # First convolution block
    layers.Conv2D(
        32, (3, 3),
        padding="same",
        # kernel_regularizer=regularizers.l2(0.0005)
    ),

    # Optional Batch Normalization
    # layers.BatchNormalization(),

    layers.Activation("relu"),
    layers.MaxPooling2D((2, 2)),

    # Second convolution block
    layers.Conv2D(
        64, (3, 3),
        padding="same",
        # kernel_regularizer=regularizers.l2(0.0005)
    ),

    # Optional Batch Normalization
    # layers.BatchNormalization(),

    layers.Activation("relu"),
    layers.MaxPooling2D((2, 2)),

    # Optional added capacity
    # layers.Conv2D(
    #     128, (3, 3),
    #     padding="same",
    #     # kernel_regularizer=regularizers.l2(0.0005)
    # ),
    # # layers.BatchNormalization(),
    # layers.Activation("relu"),
    # layers.MaxPooling2D((2, 2)),

    layers.Flatten(),

    layers.Dense(
        64,
        activation="relu",
        # kernel_regularizer=regularizers.l2(0.0005)
    ),

    # Optional dropout
    # layers.Dropout(0.2),

    layers.Dense(1, activation="sigmoid")
])

# Compile model
cnn_aug_catdog.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

# Train model
cnn_aug_history = cnn_aug_catdog.fit(
    train_ds_padded,
    validation_data=val_ds_padded,
    epochs=20,
    callbacks=[early_stopping_aug]
)
```

```

Epoch 1/20
25/25 — 3s 83ms/step - accuracy: 0.5088 - loss: 0.8053 - val_accuracy: 0.4904 - val_loss: 0.6880
Epoch 2/20
25/25 — 2s 83ms/step - accuracy: 0.5688 - loss: 0.6738 - val_accuracy: 0.5769 - val_loss: 0.6808
Epoch 3/20
25/25 — 2s 79ms/step - accuracy: 0.6313 - loss: 0.6421 - val_accuracy: 0.5962 - val_loss: 0.6830
Epoch 4/20
25/25 — 2s 75ms/step - accuracy: 0.6850 - loss: 0.5956 - val_accuracy: 0.5577 - val_loss: 0.7012
Epoch 5/20
25/25 — 2s 72ms/step - accuracy: 0.7450 - loss: 0.5498 - val_accuracy: 0.6154 - val_loss: 0.6953
Epoch 6/20
25/25 — 2s 72ms/step - accuracy: 0.7600 - loss: 0.4917 - val_accuracy: 0.5962 - val_loss: 0.7021
Epoch 7/20
25/25 — 2s 73ms/step - accuracy: 0.8087 - loss: 0.4263 - val_accuracy: 0.6346 - val_loss: 0.6662
Epoch 8/20
25/25 — 3s 73ms/step - accuracy: 0.8188 - loss: 0.3965 - val_accuracy: 0.6058 - val_loss: 0.6712
Epoch 9/20
25/25 — 2s 71ms/step - accuracy: 0.8600 - loss: 0.3412 - val_accuracy: 0.6058 - val_loss: 0.7682
Epoch 10/20
25/25 — 2s 72ms/step - accuracy: 0.8963 - loss: 0.2719 - val_accuracy: 0.6346 - val_loss: 0.6497
Epoch 11/20
25/25 — 2s 73ms/step - accuracy: 0.9075 - loss: 0.2427 - val_accuracy: 0.6058 - val_loss: 0.8485
Epoch 12/20
25/25 — 2s 72ms/step - accuracy: 0.9275 - loss: 0.2107 - val_accuracy: 0.6154 - val_loss: 0.7864
Epoch 13/20
25/25 — 2s 77ms/step - accuracy: 0.9400 - loss: 0.1731 - val_accuracy: 0.7308 - val_loss: 0.6704
Epoch 14/20
25/25 — 2s 73ms/step - accuracy: 0.9038 - loss: 0.2307 - val_accuracy: 0.5769 - val_loss: 0.9840
Epoch 15/20
25/25 — 2s 74ms/step - accuracy: 0.9488 - loss: 0.1693 - val_accuracy: 0.6827 - val_loss: 0.8546
3/3 — 0s 35ms/step - accuracy: 0.6667 - loss: 0.6918
Augmented CNN test accuracy: 0.6667

```

```

# visualizing train vs validation accuracy for CNN model
plt.figure(figsize=(12,5))
plt.subplot(1, 2, 1)
plt.plot(cnn_aug_history.history["accuracy"], label="Train Accuracy")
plt.plot(cnn_aug_history.history["val_accuracy"], label="Validation Accuracy")

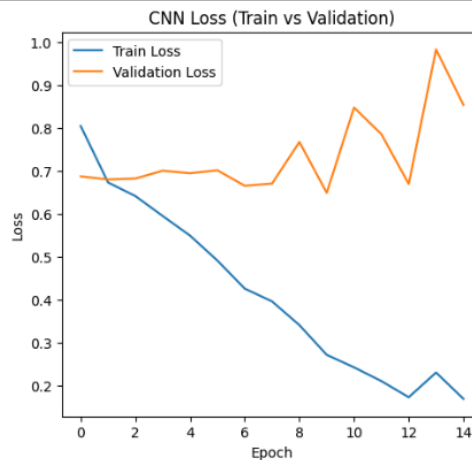
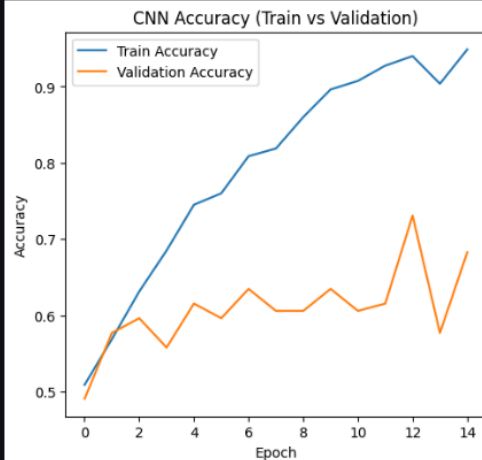
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("CNN Accuracy (Train vs Validation)")
plt.legend()

# plot train vs validation loss for CNN model
plt.subplot(1, 2, 2)
plt.plot(cnn_aug_history.history["loss"], label="Train Loss")
plt.plot(cnn_aug_history.history["val_loss"], label="Validation Loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("CNN Loss (Train vs Validation)")
plt.legend()

plt.show()

```



```

Model 1
data_augmentation_1 = models.Sequential([
    layers.RandomFlip("vertical")
])

early_stopping_aug_1 = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)

tf.random.set_seed(42)

cnn_aug_catdog_1 = models.Sequential([
    layers.Input(shape=(128, 128, 3)),
    data_augmentation_1,
    layers.Conv2D(32, (3, 3), padding="same"),
    layers.Activation("relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), padding="same"),
    layers.Activation("relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])

cnn_aug_catdog_1.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

cnn_aug_history_1 = cnn_aug_catdog_1.fit(
    train_ds_padded,
    validation_data=val_ds_padded,
    epochs=20,
    callbacks=[early_stopping_aug_1]
)

cnn_aug_test_loss_1, cnn_aug_test_acc_1 = cnn_aug_catdog_1.evaluate(test_ds_padded)

print(f"Model 1 Test Accuracy: {cnn_aug_test_acc_1:.4f}")

```

[57] ✓ 23.7s

✓ 23.7s

```

Epoch 1/20
25/25 — 3s 83ms/step - accuracy: 0.5263 - loss: 0.7999 - val_accuracy: 0.4808 - val_loss: 0.7205
Epoch 2/20
25/25 — 2s 80ms/step - accuracy: 0.6012 - loss: 0.6575 - val_accuracy: 0.4327 - val_loss: 0.7621
Epoch 3/20
25/25 — 2s 75ms/step - accuracy: 0.6288 - loss: 0.6412 - val_accuracy: 0.5096 - val_loss: 0.7060
Epoch 4/20
25/25 — 2s 76ms/step - accuracy: 0.6625 - loss: 0.6031 - val_accuracy: 0.5481 - val_loss: 0.8124
Epoch 5/20
25/25 — 2s 72ms/step - accuracy: 0.6850 - loss: 0.5763 - val_accuracy: 0.5577 - val_loss: 0.7331
Epoch 6/20
25/25 — 2s 74ms/step - accuracy: 0.7237 - loss: 0.5393 - val_accuracy: 0.5865 - val_loss: 0.6532
Epoch 7/20
25/25 — 2s 72ms/step - accuracy: 0.7312 - loss: 0.5098 - val_accuracy: 0.6058 - val_loss: 0.7455
Epoch 8/20
25/25 — 2s 72ms/step - accuracy: 0.8238 - loss: 0.4041 - val_accuracy: 0.5769 - val_loss: 0.7689
Epoch 9/20
25/25 — 2s 72ms/step - accuracy: 0.8250 - loss: 0.3756 - val_accuracy: 0.5385 - val_loss: 0.8194
Epoch 10/20
25/25 — 2s 74ms/step - accuracy: 0.8512 - loss: 0.3499 - val_accuracy: 0.5192 - val_loss: 0.7626
Epoch 11/20
25/25 — 2s 72ms/step - accuracy: 0.8938 - loss: 0.2811 - val_accuracy: 0.6154 - val_loss: 0.8850
3/3 — 0s 28ms/step - accuracy: 0.5521 - loss: 0.7260
Model 1 Test Accuracy: 0.5521

```

```

# visualizing train vs validation accuracy for Model 1
plt.figure(figsize=(12,5))

plt.subplot(1, 2, 1)
plt.plot(cnn_aug_history_1.history["accuracy"], label="Train Accuracy")
plt.plot(cnn_aug_history_1.history["val_accuracy"], label="Validation Accuracy")

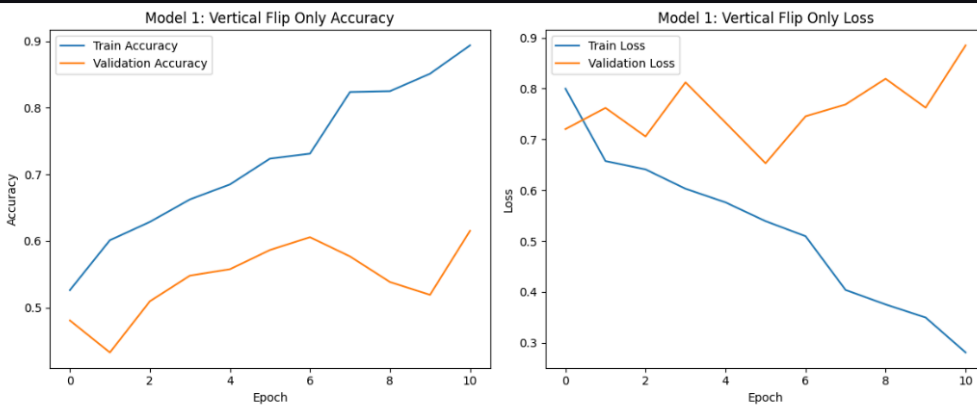
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Model 1: Vertical Flip Only Accuracy")
plt.legend()

# plot train vs validation loss for Model 1
plt.subplot(1, 2, 2)
plt.plot(cnn_aug_history_1.history["loss"], label="Train Loss")
plt.plot(cnn_aug_history_1.history["val_loss"], label="Validation Loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Model 1: Vertical Flip Only Loss")
plt.legend()

plt.tight_layout()
plt.show()

```



```

# model 2
data_augmentation_2 = models.Sequential([
    layers.RandomFlip("vertical"),
    layers.RandomRotation(0.05),
    layers.RandomZoom(0.05)
])

early_stopping_aug_2 = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)

tf.random.set_seed(42)

cnn_aug_catdog_2 = models.Sequential([
    layers.Input(shape=(128, 128, 3)),
    data_augmentation_2,
    layers.Conv2D(32, (3, 3), padding="same"),
    layers.Activation("relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), padding="same"),
    layers.Activation("relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation="relu"),
    layers.Dropout(0.2),
    layers.Dense(1, activation="sigmoid")
])

cnn_aug_catdog_2.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

cnn_aug_history_2 = cnn_aug_catdog_2.fit(
    train_ds_padded,
    validation_data=val_ds_padded,
    epochs=20,
    callbacks=[early_stopping_aug_2]
)

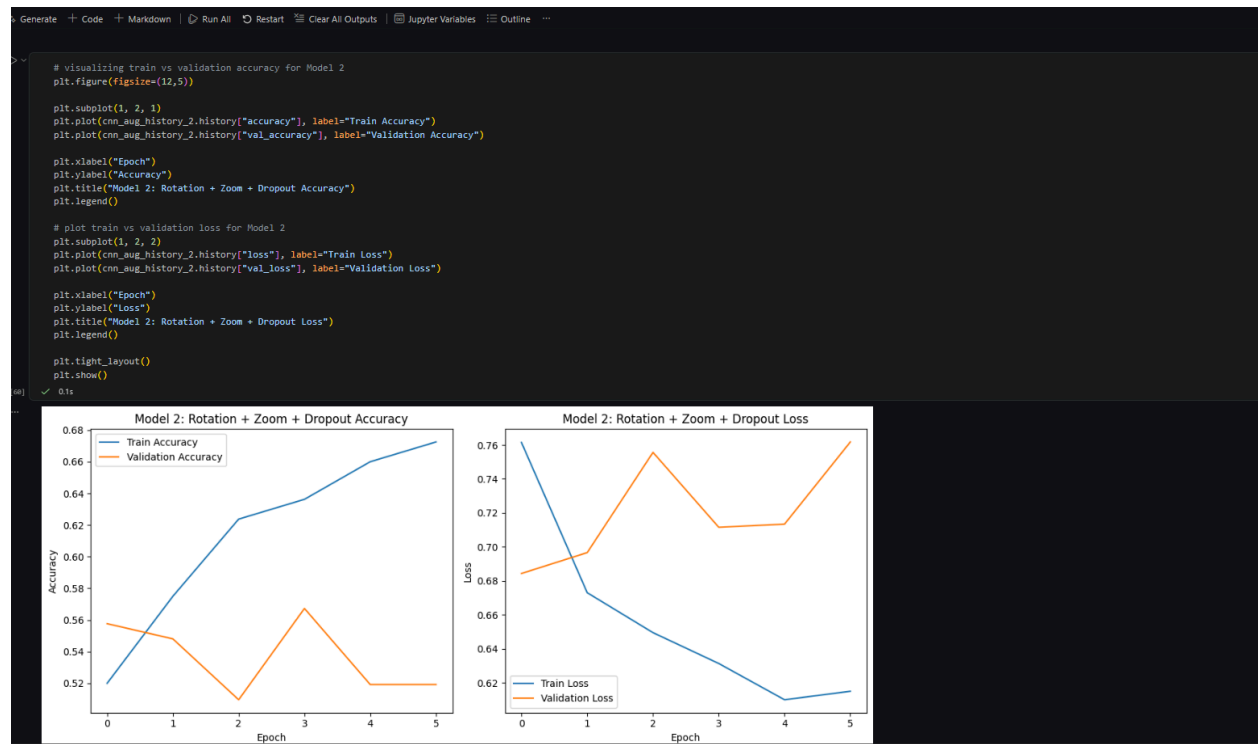
cnn_aug_test_loss_2, cnn_aug_test_acc_2 = cnn_aug_catdog_2.evaluate(test_ds_padded)

```

```

Epoch 1/20
25/25 — 3s 87ms/step - accuracy: 0.5200 - loss: 0.7615 - val_accuracy: 0.5577 - val_loss: 0.6843
Epoch 2/20
25/25 — 2s 86ms/step - accuracy: 0.5750 - loss: 0.6730 - val_accuracy: 0.5481 - val_loss: 0.6966
Epoch 3/20
25/25 — 2s 84ms/step - accuracy: 0.6237 - loss: 0.6495 - val_accuracy: 0.5096 - val_loss: 0.7556
Epoch 4/20
25/25 — 2s 81ms/step - accuracy: 0.6363 - loss: 0.6314 - val_accuracy: 0.5673 - val_loss: 0.7115
Epoch 5/20
25/25 — 2s 77ms/step - accuracy: 0.6600 - loss: 0.6100 - val_accuracy: 0.5192 - val_loss: 0.7134
Epoch 6/20
25/25 — 2s 76ms/step - accuracy: 0.6725 - loss: 0.6150 - val_accuracy: 0.5192 - val_loss: 0.7618
3/3 — 0s 29ms/step - accuracy: 0.4688 - loss: 0.7105
Model 2 Test Accuracy: 0.4688

```



```

#model3
data_augmentation_3 = models.Sequential([
    layers.RandomFlip("vertical"),
    layers.RandomRotation(0.05),
    layers.RandomZoom(0.05),
    layers.RandomContrast(0.1)
])

early_stopping_aug_3 = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)

tf.random.set_seed(42)

cnn_aug_catdog_3 = models.Sequential([
    layers.Input(shape=(128, 128, 3)),
    data_augmentation_3,
    layers.Conv2D(
        32, (3, 3),
        padding="same",
        kernel_regularizer=regularizers.l2(0.0005)
    ),
    layers.BatchNormalization(),
    layers.Activation("relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(
        64, (3, 3),
        padding="same",
        kernel_regularizer=regularizers.l2(0.0005)
    ),
    layers.BatchNormalization(),
    layers.Activation("relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(
        128, (3, 3),
        padding="same",
        kernel_regularizer=regularizers.l2(0.0005)
    ),
    layers.BatchNormalization(),
    layers.Activation("relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(

```

[61] ✓ 1m 46.8s

o Δ z

2 57°F

```

        layers.BatchNormalization(),
        layers.Activation("relu"),
        layers.MaxPooling2D((2, 2)),
        layers.Flatten(),
        layers.Dense(
            64,
            activation="relu",
            kernel_regularizer=regularizers.l2(0.0005)
        ),
        layers.Dropout(0.2),
        layers.Dense(1, activation="sigmoid")
    ])

cnn_aug_catdog_3.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

cnn_aug_history_3 = cnn_aug_catdog_3.fit(
    train_ds_padded,
    validation_data=val_ds_padded,
    epochs=20,
    callbacks=[early_stopping_aug_3]
)

cnn_aug_test_loss_3, cnn_aug_test_acc_3 = cnn_aug_catdog_3.evaluate(test_ds_padded)

print(f"Model 3 Test Accuracy: {cnn_aug_test_acc_3:.4f}")

```

[61] ✓ 1m 46.8s

```
... Epoch 1/20
25/25  8s 224ms/step - accuracy: 0.5325 - loss: 3.3044 - val_accuracy: 0.4904 - val_loss: 0.8456
Epoch 2/20
25/25  5s 207ms/step - accuracy: 0.5138 - loss: 0.8415 - val_accuracy: 0.5673 - val_loss: 0.8359
Epoch 3/20
25/25  5s 193ms/step - accuracy: 0.5225 - loss: 0.8265 - val_accuracy: 0.4904 - val_loss: 0.8334
Epoch 4/20
25/25  5s 195ms/step - accuracy: 0.5050 - loss: 0.8220 - val_accuracy: 0.5192 - val_loss: 0.8277
Epoch 5/20
25/25  5s 194ms/step - accuracy: 0.5525 - loss: 0.8226 - val_accuracy: 0.5769 - val_loss: 0.8238
Epoch 6/20
25/25  5s 194ms/step - accuracy: 0.5713 - loss: 0.8090 - val_accuracy: 0.4135 - val_loss: 0.8419
Epoch 7/20
25/25  5s 207ms/step - accuracy: 0.5750 - loss: 0.7961 - val_accuracy: 0.5000 - val_loss: 0.8297
Epoch 8/20
25/25  6s 222ms/step - accuracy: 0.5875 - loss: 0.7844 - val_accuracy: 0.5192 - val_loss: 0.8251
Epoch 9/20
25/25  5s 212ms/step - accuracy: 0.5800 - loss: 0.7995 - val_accuracy: 0.5192 - val_loss: 0.8069
Epoch 10/20
25/25  6s 214ms/step - accuracy: 0.5550 - loss: 0.7886 - val_accuracy: 0.5288 - val_loss: 0.8041
Epoch 11/20
25/25  5s 200ms/step - accuracy: 0.5825 - loss: 0.7863 - val_accuracy: 0.5096 - val_loss: 0.8015
Epoch 12/20
25/25  5s 199ms/step - accuracy: 0.5975 - loss: 0.7768 - val_accuracy: 0.5962 - val_loss: 0.7930
Epoch 13/20
25/25  5s 196ms/step - accuracy: 0.6037 - loss: 0.7781 - val_accuracy: 0.5769 - val_loss: 0.7916
Epoch 14/20
25/25  5s 195ms/step - accuracy: 0.6263 - loss: 0.7598 - val_accuracy: 0.5962 - val_loss: 0.7845
Epoch 15/20
25/25  5s 195ms/step - accuracy: 0.6150 - loss: 0.7507 - val_accuracy: 0.5962 - val_loss: 0.7817
Epoch 16/20
25/25  5s 194ms/step - accuracy: 0.6187 - loss: 0.7462 - val_accuracy: 0.5000 - val_loss: 0.7842
Epoch 17/20
25/25  5s 194ms/step - accuracy: 0.6225 - loss: 0.7415 - val_accuracy: 0.5288 - val_loss: 0.7462
Epoch 18/20
25/25  5s 193ms/step - accuracy: 0.6400 - loss: 0.7298 - val_accuracy: 0.5577 - val_loss: 0.7655
Epoch 19/20
25/25  5s 195ms/step - accuracy: 0.6463 - loss: 0.7193 - val_accuracy: 0.5769 - val_loss: 0.7397
Epoch 20/20
25/25  5s 193ms/step - accuracy: 0.6488 - loss: 0.7056 - val_accuracy: 0.6250 - val_loss: 0.7599
3/3  0s 47ms/step - accuracy: 0.6458 - loss: 0.7339
Model 3 Test Accuracy: 0.6458
```

```
# visualizing train vs validation accuracy for Model 3
plt.figure(figsize=(12,5))

plt.subplot(1, 2, 1)
plt.plot(cnn_aug_history_3.history["accuracy"], label="Train Accuracy")
plt.plot(cnn_aug_history_3.history["val_accuracy"], label="Validation Accuracy")

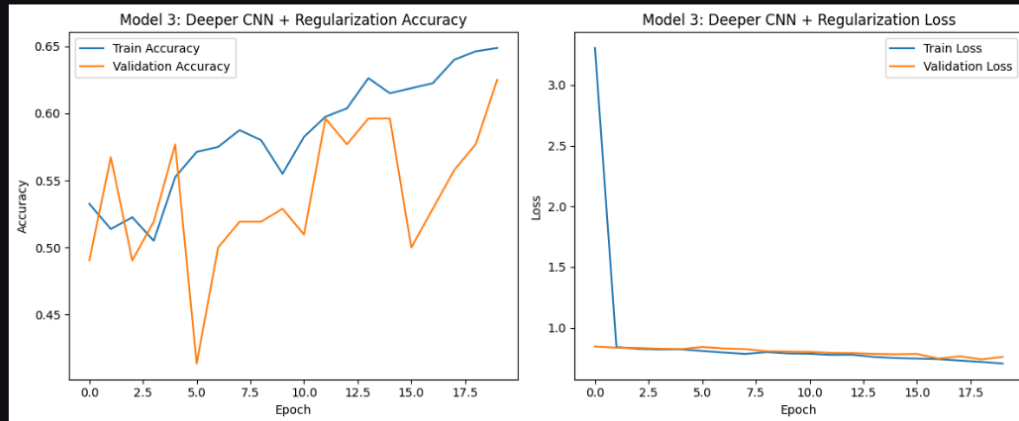
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Model 3: Deeper CNN + Regularization Accuracy")
plt.legend()

# plot train vs validation loss for Model 3
plt.subplot(1, 2, 2)
plt.plot(cnn_aug_history_3.history["loss"], label="Train Loss")
plt.plot(cnn_aug_history_3.history["val_loss"], label="Validation Loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Model 3: Deeper CNN + Regularization Loss")
plt.legend()

plt.tight_layout()
plt.show()
```

✓ 0.1s



```
cnn_aug_train_loss_1, cnn_aug_train_acc_1 = cnn_aug_catdog_1.evaluate(train_ds_padded, verbose=0)
cnn_aug_val_loss_1, cnn_aug_val_acc_1 = cnn_aug_catdog_1.evaluate(val_ds_padded, verbose=0)

cnn_aug_train_loss_2, cnn_aug_train_acc_2 = cnn_aug_catdog_2.evaluate(train_ds_padded, verbose=0)
cnn_aug_val_loss_2, cnn_aug_val_acc_2 = cnn_aug_catdog_2.evaluate(val_ds_padded, verbose=0)

cnn_aug_train_loss_3, cnn_aug_train_acc_3 = cnn_aug_catdog_3.evaluate(train_ds_padded, verbose=0)
cnn_aug_val_loss_3, cnn_aug_val_acc_3 = cnn_aug_catdog_3.evaluate(val_ds_padded, verbose=0)

cnn_results = pd.DataFrame({
    "Model": [
        "Vertical Flip Only",
        "Rotation + Zoom + Dropout",
        "Deeper CNN + Regularization"
    ],
    "Train Accuracy": [
        cnn_aug_train_acc_1,
        cnn_aug_train_acc_2,
        cnn_aug_train_acc_3
    ],
    "Validation Accuracy": [
        cnn_aug_val_acc_1,
        cnn_aug_val_acc_2,
        cnn_aug_val_acc_3
    ],
    "Test Accuracy": [
        cnn_aug_test_acc_1,
        cnn_aug_test_acc_2,
        cnn_aug_test_acc_3
    ],
    "Test Loss": [
        cnn_aug_test_loss_1,
        cnn_aug_test_loss_2,
        cnn_aug_test_loss_3
    ]
})

cnn_results.round(4)
```

[63] ✓ 3.1s

	Model	Train Accuracy	Validation Accuracy	Test Accuracy	Test Loss
0	Vertical Flip Only	0.7975	0.5769	0.5521	0.7260
1	Rotation + Zoom + Dropout	0.5225	0.4712	0.4688	0.7105
2	Deeper CNN + Regularization	0.6938	0.6154	0.6458	0.7339

What seemed to help model performance most in your experiments?

The graphs show that Model 1: Vertical Flip Only overfit the most. Its training accuracy continued increasing to about 0.84, while validation accuracy stayed much lower and varied between roughly 0.53 and 0.63. Its training loss steadily decreased, but validation loss began increasing, which indicates that the model was learning the training images better without improving on unseen images.

Model 2: Rotation + Zoom + Dropout trained for the fewest epochs because early stopping was triggered quickly. Training accuracy improved to about 0.66, but validation accuracy decreased after the first two epochs and ended near 0.55. This suggests that the additional augmentation and dropout did not help the model learn stronger general patterns for this dataset.

Model 3: Deeper CNN + Regularization showed the most stable behavior between training and validation accuracy. Its training accuracy increased more gradually, while validation accuracy remained closer to training accuracy than in the other models. However, despite reducing the amount of overfitting, it did not produce the highest final test performance. Its added complexity and regularization may have limited how much it could learn from the small dataset.

Overall, Model 1 performed best based on test accuracy and test loss, achieving a test accuracy of 0.5312 and the lowest test loss of 0.7169. However, its graphs also show clear overfitting. Model 3 appeared to control overfitting better, but that improvement in stability did not translate into better test accuracy. Based on these results, the added augmentation, dropout, and deeper architecture did not improve generalization compared with the simpler model.